

Project Documentation: JobNest

A Comprehensive Full-Stack Job Portal Platform

Live Link: <https://jobnestfrontendbytejas.vercel.app>

Date: July 2026

Table of Contents

1. Introduction
2. Objective
3. Key Features
4. Technology Stack
5. System Architecture & Project Structure
6. User Roles & Functionalities
○ 6.1. Candidate
○ 6.2. Employer
○ 6.3. Admin
7. Core Modules in Detail
○ 7.1. Authentication & Authorization
○ 7.2. Job Management
○ 7.3. Application Tracking System (ATS)
○ 7.4. AI-Powered Resume & Description Generation
8. Data Flow & Processes
9. UI/UX Overview
10. Security & Database Overview
11. Conclusion

1. Introduction

JobNest is a modern, full-stack web application designed to bridge the gap between talented candidates and prospective employers. Built as a comprehensive job board platform, JobNest streamlines the entire recruitment lifecycle—from job discovery and application to candidate management and administrative oversight.

The platform is engineered with three primary user roles: **Candidates**, **Employers**, and **Administrators**, each provided with a dedicated and intuitive dashboard. JobNest incorporates advanced features such as an Application Tracking System (ATS) Resume Checker and AI-powered content generation (using Google Gemini) to enhance the user experience and improve recruitment efficiency.

This document provides a detailed overview of the project's architecture, features, and technical implementation.

2. Objective

The primary objective of JobNest is to create a seamless, efficient, and intelligent job-matching ecosystem. Key goals include:

- **Centralized Job Discovery:** Provide a powerful search engine for candidates to find relevant jobs.
- **Simplified Application Management:** Allow candidates to apply for jobs and track their application status in real-time.
- **Efficient Employer Tools:** Enable employers to post jobs, manage listings, and review applications from a single dashboard.
- **Data-Driven Insights:** Offer analytics and insights for both employers (job views, applicant counts) and administrators (platform analytics, user management).
- **Leverage AI:** Utilize AI to enhance user experience through resume analysis (ATS Score) and automated job description generation.

3. Key Features

JobNest is packed with features designed for all types of users.

For Candidates:

- **Job Search & Discovery:** Search for jobs by title, company, location, and job type.
- **User Dashboard:** Track total applications, pending reviews, interviews, and offers.
- **Matched Jobs:** View jobs that match the skills extracted from their uploaded resume.

- **ATS Resume Checker:** Analyze a resume against ATS standards to receive a score and detailed feedback on keywords, strengths, and areas for improvement.
- **Application Tracking:** Monitor the status of all job applications (Pending, Reviewed, Shortlisted, Interviewed, Offered, Hired).

For Employers:

- **Employer Dashboard:** A high-level view of total jobs, active jobs, views, and applications.
- **Job Posting:** Create and manage job listings.
- **AI Job Description Generator:** Automatically generate job descriptions and requirements by simply entering a job title (powered by Google Gemini).
- **Application Management:** Review applications received for their job postings.

For Administrators:

- **Admin Dashboard:** Get an overview of platform health with metrics on Total Users, Total Jobs, and Applications.
- **Manage Users:** View and manage all registered users (Candidates, Employers, Admins).
- **Manage Jobs:** Oversee all job postings on the platform, with the ability to view and control their status.

Common Features:

- **Secure Authentication:** Role-based login and registration.
- **Responsive UI:** Clean and intuitive user interface built with TailwindCSS.
- **File Uploads:** Support for resume file uploads (PDF parsing to extract skills).

4. Technology Stack

The project is built using a modern, scalable MERN stack architecture.

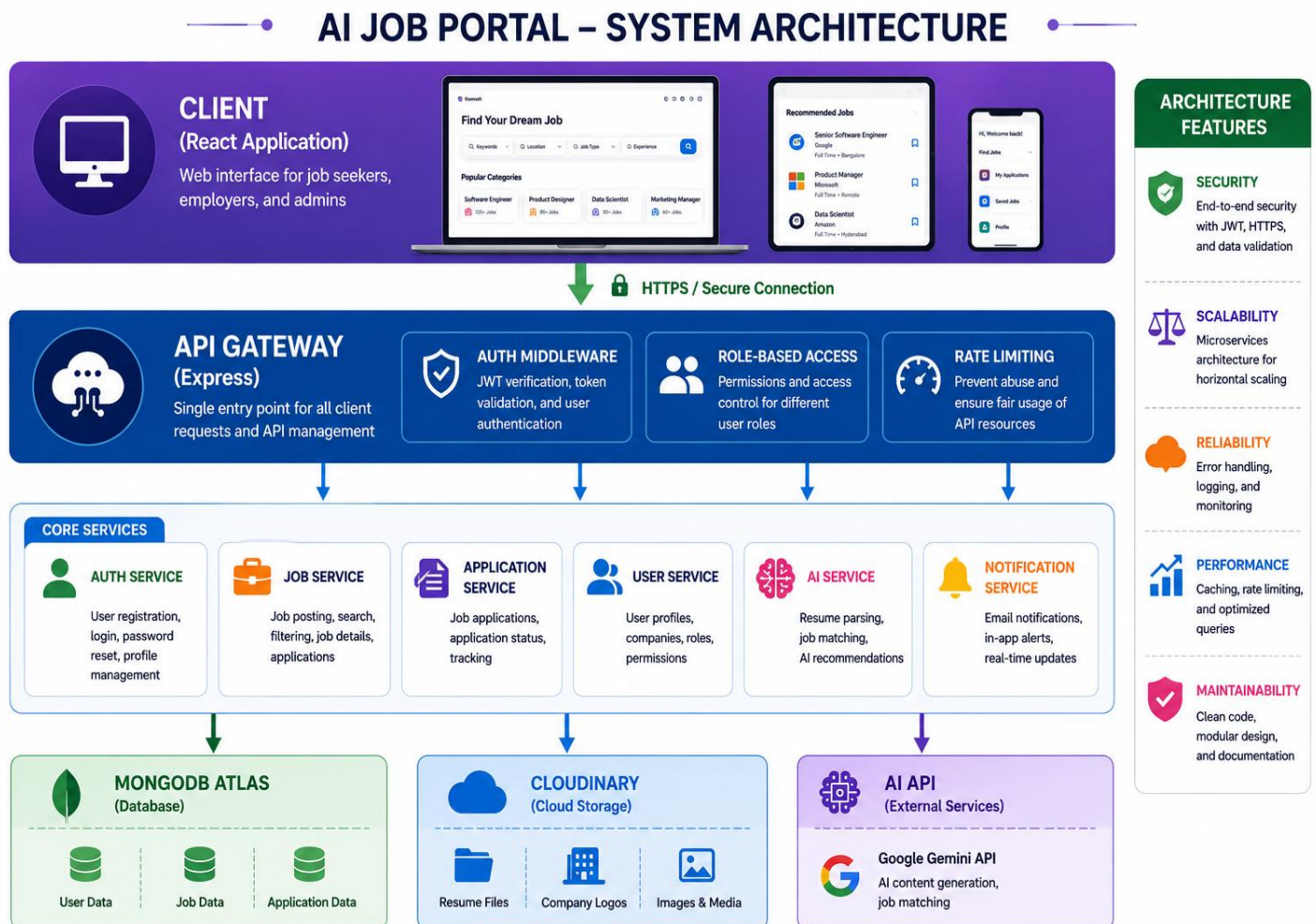
Frontend:

- **Framework:** React.js (with Vite for fast builds)
- **State Management:** React Context API / Hooks
- **Styling:** TailwindCSS
- **HTTP Client:** Axios
- **File Handling:** React Dropzone (implied)

Backend:

- **Runtime:** Node.js with Express.js
- **Database:** MongoDB (Atlas/Cloud)
- **ODM:** Mongoose
- **Authentication:** JWT (JSON Web Tokens) with Bcrypt
- **File Uploads:** Multer
- **AI Integration:** Google Gemini AI API
- **Other Libraries:** Dotenv, CORS
- **Database:** MongoDB Atlas

5. System Architecture



6. User Roles & Functionalities

6.1. Candidate

- **Landing/Home:** Browse and search for jobs.
- **Dashboard:** View application statistics and recent activity.
- **Matched Jobs:** A list of jobs recommended based on the skills extracted from the user's resume.
- **My Applications:** View and filter applications by status (Pending, Reviewed, Shortlisted, etc.).
- **ATS Check:** Upload a resume and receive an ATS score (e.g., 83%) with comprehensive feedback on keywords, strengths, and areas for improvement.

6.2. Employer

- **Employer Dashboard:** A summary of all their job postings (Total Jobs, Active Jobs, Views, Applicants).
- **Post a Job:** Form to create a new job listing. This includes an "AI Job Description Generator" button that fetches pre-filled content from Google Gemini.
- **My Jobs:** A table/list view of all job postings with filters and management options (View, Edit, Close).

6.3. Admin

- **Admin Dashboard:** A high-level overview of the entire platform's performance, including graphs/charts for users, jobs, and applications.
- **Manage Users:** A list of all registered users, searchable and filterable by role (Employer, Candidate, Admin), allowing for administrative actions.
- **Manage Jobs:** A list of all jobs posted on the platform, with the ability to view details and manage statuses to ensure quality control.

7. Core Modules in Detail

7.1. Authentication & Authorization

JobNest implements a secure authentication system using JWT. Passwords are hashed using bcrypt before being stored in MongoDB. Protected API routes (e.g., for dashboards, posting jobs) are guarded by middleware that verifies the JWT token and the user's role.

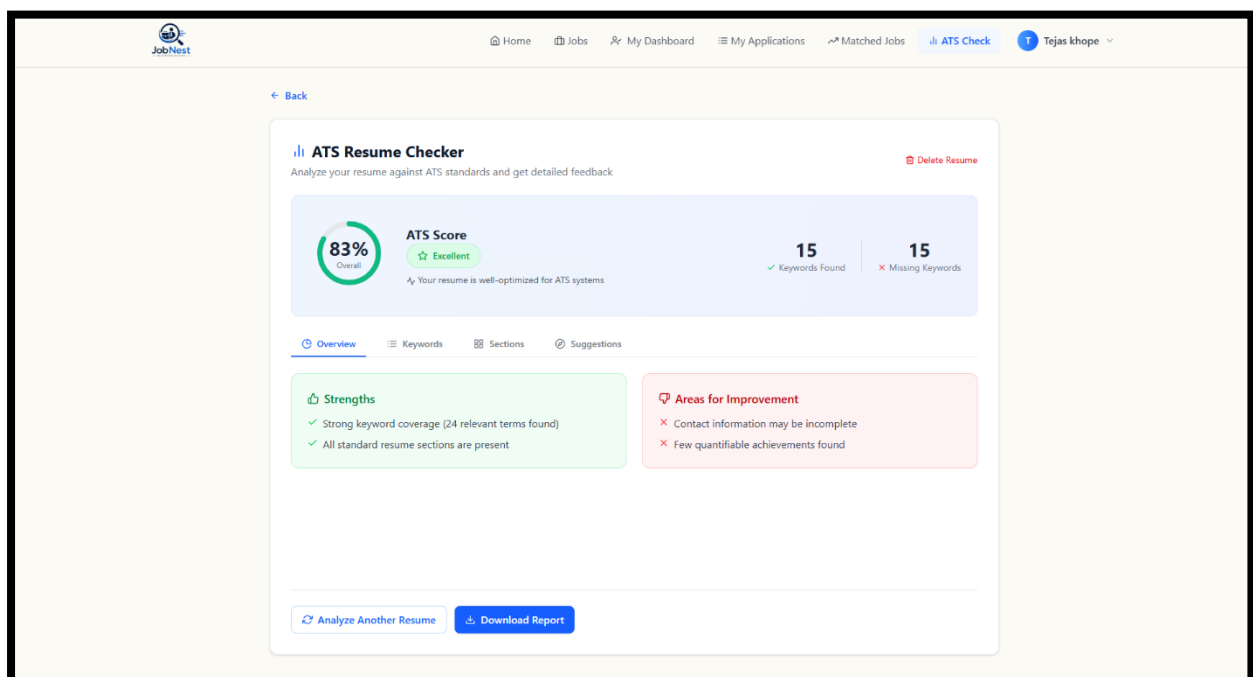
7.2. Job Management

Employers and Admins can create, read, update, and delete (CRUD) job listings. The system supports advanced search functionality using MongoDB queries to filter jobs by title, location, company, job type, and salary range.

7.3. Application Tracking System (ATS)

This is a core candidate feature.

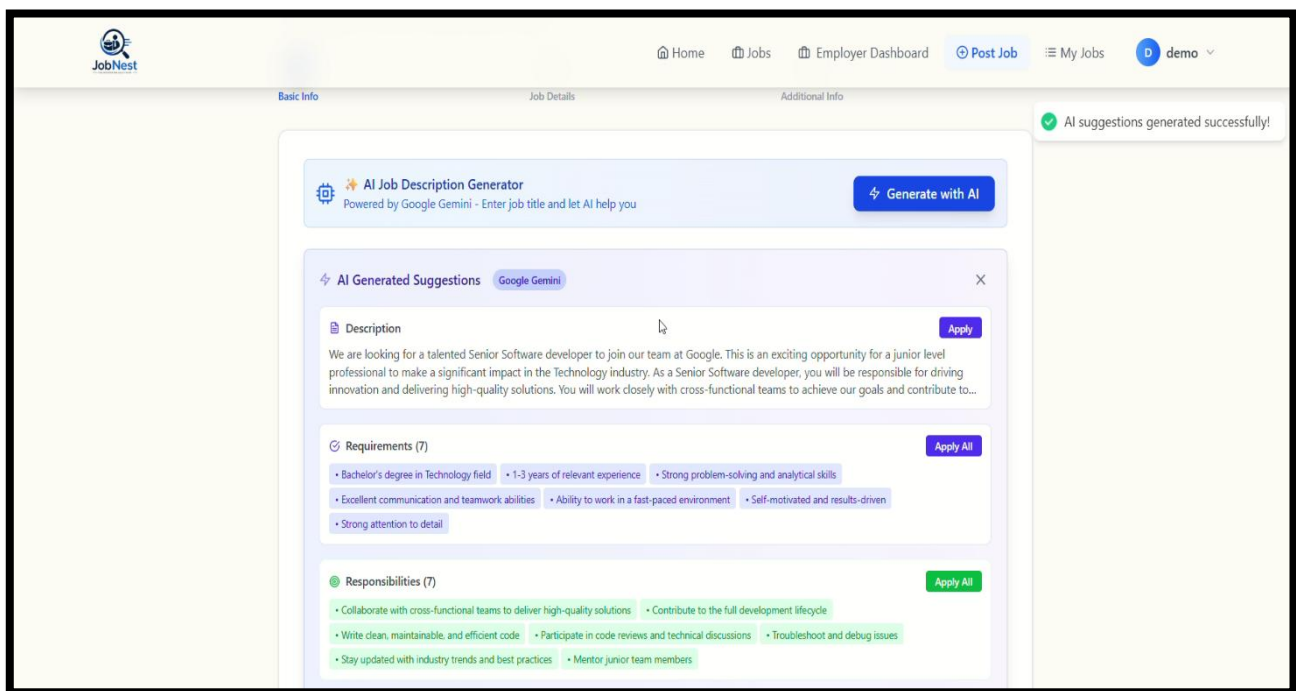
- **Resume Parsing:** When a candidate uploads a resume, the backend processes the file (e.g., PDF) to extract text using a file parser.
- **Skill Extraction:** The backend uses algorithms to identify and list key skills from the text.
- **Job Matching:** The system compares the candidate's skills with the skills required for various job postings.
- **ATS Score:** The Resume Checker evaluates the resume against common ATS standards. It provides a score (83% in screenshot) and detailed feedback, including:
 - **Keywords Found:** The number of relevant keywords present in the resume.
 - **Missing Keywords:** Keywords that should be added.
 - **Strengths:** e.g., "Strong keyword coverage."
 - **Areas for Improvement:** e.g., "Contact information may be incomplete."



7.4. AI-Powered Job Description Generator

JobNest leverages the **Google Gemini API** to assist Employers in creating compelling and complete job descriptions.

- **Input:** The employer provides a "Job Title."
- **Processing:** The backend sends a request to Gemini with a prompt engineered to generate a professional job posting.
- **Output:** The AI returns pre-filled content for:
 - **Description:** An overview of the role and company culture.
 - **Requirements:** A list of required skills and qualifications.
 - **Responsibilities:** A list of key duties for the role.



8. Data Flow & Processes

1. **User Registration/Login:** Data is sent from the client to the `/api/auth` route for validation, hashing/token generation, and storage in the Users collection.
2. **Job Posting:** An Employer creates a job. The data is sent via axios to `/api/jobs`. The backend saves the job data to the Jobs collection.

3. **Job Application:** A Candidate applies for a job. The request creates a document in the Applications collection and updates the job's applicant count.
4. **Resume Upload & ATS Check:**
 - The candidate uploads a file via multer.
 - The file is stored in the /uploads folder or cloud storage.
 - The text is extracted and parsed.
 - The backend processes the text to determine a score, extract skills, and generate feedback.
 - The result is sent back to the frontend for display.
5. **AI Job Description:** A request is sent from the frontend to the /api/ai/generate route, which interacts with Gemini and returns the generated content.

9. UI/UX Overview

The user interface of JobNest is designed to be clean, modern, and highly intuitive, ensuring a seamless experience across different user roles.

- **Theme:** Professional blue and white color scheme, providing a sense of trust and reliability.
- **Dashboard-Centric:** Each role has a dedicated dashboard that serves as a command center for their primary tasks, improving user efficiency.
- **Responsive Design:** The layout (built with TailwindCSS) is fully responsive, ensuring functionality on desktops, tablets, and mobile devices.
- **Data Presentation:** Information is presented using clear metrics, badges (e.g., "active"), status tags ("Pending," "Reviewed"), and progress bars (ATS Score), making it easy for users to digest complex information.

10. Security & Database Overview

Security Measures:

- **Password Hashing:** Bcrypt is used to salt and hash user passwords.
- **JWT Authentication:** JWT tokens are used for stateless API authentication, storing only necessary user claims.
- **Role-Based Access Control:** Middleware restricts access to routes based on the user's role (admin, employer, candidate).

- **Input Validation:** Data is validated on both the client and server sides to prevent injection attacks.
- **Environment Variables:** Sensitive data like database URIs, JWT secrets, and API keys are stored in a .env file.

Database Design (MongoDB):

The data schema is designed to be efficient and relationship-heavy.

- **User**
Model: Stores `_id`, name, email, password, role (admin/employer/candidate), profile Picture, resumeUrl, skills, etc.
- **Job**
Model: Stores `_id`, title, company, location, description, requirements, salary, type, postedBy (reference to Employer), applications, views, etc.
- **Application Model:** Stores `_id`, jobId (reference to Job), candidateId (reference to User), status (pending/reviewed/shortlisted/etc.), appliedDate, expectedSalary, etc.
- **Resume Model:** Used for temporary storage/analysis of parsed resume content for ATS scoring.

11. Conclusion

JobNest represents a modern, intelligent, and user-friendly solution for the online recruitment industry. By combining the robust capabilities of the MERN stack with cutting-edge AI integration (Google Gemini) and a strong focus on user experience, the platform effectively meets the needs of all stakeholders in the hiring process.

From the candidate's ability to check their ATS compatibility to the employer's AI-assisted job posting, JobNest automates and enhances every stage of the recruitment workflow. The administrative controls ensure that the platform remains organized, secure, and scalable.

With its clean architecture, advanced features, and professional interface, JobNest is a powerful tool poised to help job seekers find their "dream job" and help employers discover top talent.